

1. Informe Técnico: Fuerza Bruta y Backtracking

Diego Alejandro Angarita
Emmanuel Castaño Sepúlveda
Luis Miguel Mira Mejía

Escuela de Ciencias Aplicadas e Ingeniería, Universidad EAFIT

Modelación y Simulación V

Ing. Carlos Alberto Álvarez Henao

23 de agosto de 2026

2. Introducción

El presente informe documenta el desarrollo y análisis empírico de dos paradigmas algorítmicos aplicados al dominio de las contraseñas y los espacios de cadenas de texto: la enumeración exhaustiva y la construcción incremental con poda.

El proyecto aborda la exploración de un espacio de búsqueda desde dos perspectivas complementarias. En el primer módulo, bajo el paradigma de Fuerza Bruta, se modela el proceso de un analista de seguridad que busca descifrar una contraseña desconocida a partir de su hash SHA-256, evaluando sistemáticamente todo el espacio de combinaciones posibles frente a un ataque alternativo por diccionario. En el segundo módulo, mediante Backtracking, se asume el rol de un diseñador de políticas de seguridad para generar contraseñas válidas que cumplan con reglas estrictas de composición y longitud, evitando generar previamente todas las cadenas posibles.

En ese sentido, el propósito central de esta práctica es experimentar y cuantificar la distinción conceptual entre explorar un problema sin ninguna estructura subyacente (Fuerza Bruta) frente a explorar de forma iterativa descartando ramas que no conducen a soluciones factibles (Backtracking). Esto permite evidenciar empíricamente la barrera computacional impuesta por el crecimiento exponencial de las combinaciones y medir el porcentaje exacto de reducción del espacio de búsqueda que se logra al aplicar condiciones tempranas de poda.

De tal manera, el presente informe inicia presentando el contexto y la fundamentación teórica que sustentan cada paradigma. Posteriormente, se define el modelamiento formal de los estados de búsqueda, seguido por el diseño algorítmico, el pseudocódigo y los detalles más relevantes de la implementación en C++17. La sección final condensa el análisis de complejidad teórica, los casos de prueba ejecutados y los resultados de experimentación. A partir de estos datos, se establece una comparación directa entre el rendimiento de las diferentes aproximaciones y el contraste entre el crecimiento teórico y las mediciones observadas.

3. Contexto Del Problema

El problema se aborda desde dos escenarios complementarios.

En el Módulo de Fuerza Bruta (FB), el objetivo es recuperar una contraseña a partir de su hash criptográfico SHA-256. Debido a que estas funciones están diseñadas para no ser reversibles, la única forma de encontrar la credencial es evaluar sistemáticamente todo el espacio de combinaciones posibles. Esto evidencia en la práctica por qué las políticas de contraseñas son necesarias: el costo temporal crece de forma exponencial al aumentar la longitud, hasta llegar a un punto donde el cómputo se vuelve intratable.

En el Módulo de Backtracking (BT), el escenario cambia al diseño de una política de contraseñas. El objetivo es generar únicamente las cadenas que cumplen con reglas

ENTREGA I

específicas de seguridad. En lugar de generar todo el espacio de cadenas para luego filtrarlas, la construcción se hace incrementalmente carácter por carácter. Si un prefijo parcial ya no puede cumplir la política, la búsqueda retrocede inmediatamente descartando esa rama (poda). Esto reduce drásticamente los nodos visitados, aunque la complejidad en el peor caso siga siendo exponencial.

4. Fundamentación Teórica.

Por lo que se refiere a Fuerza Bruta, este método explora sistemáticamente todo el espacio de soluciones sin usar propiedades estructurales para descartar candidatos. En nuestra instancia, el espacio total para contraseñas de longitud n sobre un alfabeto Σ es exactamente $|\Sigma|^n$. La complejidad temporal es exponencial $O(|\Sigma|^n)$, asumiendo un costo $O(1)$ por cada cálculo del hash SHA-256. La complejidad espacial se acota a $O(n)$ al generar exhaustivamente un candidato a la vez en memoria.

Por otra parte, Backtracking construye soluciones de forma incremental sobre un árbol de estados parciales. Tal como formaliza Skiena (2020), esta técnica optimiza la búsqueda exhaustiva al evaluar tempranamente las condiciones de factibilidad, podando ramas infactibles tan pronto se detectan para evitar recorrer subárboles completos. En este diseño, el estado parcial es el prefijo de la contraseña. Si el prefijo ya no tiene espacio para cumplir con las reglas de la política (ej. mínimo de dígitos), la rama se poda. Aunque el peor caso temporal sigue siendo $O(|\Sigma|^n)$, la poda reduce drásticamente los nodos visitados. La complejidad espacial es $O(n)$, correspondiente a la pila de recursión.

Concepto teórico	Cómo se ve en <code>BT_Angarita.cpp</code>
Estado parcial	El prefijo <code>password[0..pos-1]</code> , la posición actual <code>pos</code> , y los cuatro contadores <code>nLower</code> , <code>nUpper</code> , <code>nDigit</code> , <code>nSymbol</code> de clases ya colocadas.
Condición de factibilidad	<code>faltan <= pol.n - pos</code> : los mínimos que aún faltan caben en las posiciones que quedan. Si se viola, <code>return</code> (poda).
Poda	La línea <code>if (faltan > pol.n - pos) return;</code> y la regla <code>if (... c == password[pos-1]) continue;</code> . La primera corta subárboles imposibles; la segunda evita ramas prohibidas por la política.
Árbol de búsqueda	Raíz = cadena vacía (<code>pos = 0</code>). Cada nodo tiene hasta 67 hijos (un carácter del alfabeto). Las hojas están en profundidad <code>n</code> . Una hoja es solución si el prefijo completo cumple todos los mínimos.

ENTREGA I

Backtracking no mejora el orden de complejidad en el peor caso. El peor caso sigue siendo $O(67^n)$: si la política es tan permisiva que ninguna rama puede podarse antes de la hoja, el árbol se recorre completo. Lo que backtracking cambia es el número de nodos efectivamente visitados en la práctica: cuando la política es restrictiva, la poda elimina subárboles enteros y el recorrido real es órdenes de magnitud menor que 67^n . Es una mejora empírica, no asintótica.

5. Modelamiento

5.1. Fuerza Bruta:

5.1.1. Modelamiento del problema.

Para un alfabeto Σ

y una longitud n , el espacio de búsqueda es:

$$\Sigma^n = \{x_1x_2 \dots x_n \mid x_i \in \Sigma\}$$

Los alfabetos utilizados son:

$$A_1 = \{a, b, \dots, z\}, \text{ con } |\Sigma| = 26.$$

$$A_2 = \{a, b, \dots, z, 0, 1, \dots, 9\}, \text{ con } |\Sigma| = 36.$$

La función de verificación es:

$$V(c, H) = \begin{cases} 1 & \text{si } SHA256(c) = H \\ 0 & \text{en otro caso} \end{cases}$$

5.1.2. Definición de entradas y salidas.

Entrada:

- Hash objetivo SHA-256 de 64 caracteres hexadecimales.
- Alfabeto A1 o A2.
- Longitud mínima y máxima.

Salida:

- Contraseña en texto plano si se encuentra.
- Estado *no encontrada*.
- Cantidad de candidatos evaluados.
- Tiempo de ejecución en milisegundos.

5.1.3. Definición de las restricciones.

El número de candidatos para una longitud fija es:

$$|\Sigma|^n$$

Para longitudes crecientes desde 1 hasta N :

$$\sum_{i=1}^N |\Sigma|^i$$

Ejemplo:

Alfabeto A1: $n=3$ (17.576 candidatos), $n=4$ (456.976 candidatos), $n=5$ (11.881.376 candidatos).

Alfabeto A2: $n=3$ (46.656 candidatos), $n=4$ (1.679.616 candidatos), $n=5$ (60.466.176 candidatos), $n=6$ (2.176.782.336 candidatos).

Nota: En un computador personal, A2 con longitudes de 6 caracteres puede resultar costoso. El experimento debe limitarse a longitudes pequeñas o detenerse al encontrar la contraseña.

5.1.4. Diseño del algoritmo.

El algoritmo representa cada candidato como un número en base

$$|\Sigma|$$

El primer candidato es:

aaaaa

Después se incrementa la última posición. Cuando se alcanza el último símbolo, se reinicia esa posición y se incrementa la anterior:

aaaaa

aaaab

aaaac

...

aaaaz

aaa a0

...

La implementación solo mantiene la cadena actual, por lo que utiliza espacio

$$O(n)$$

5.1.5. Pseudocódigo.

FUERZA_BRUTA(hashObjetivo, alfabeto, longitudMinima, longitudMaxima)

```
candidatosEvaluados <- 0
```

```
PARA longitud DESDE longitudMinima HASTA longitudMaxima HACER  
  candidato <- cadena de longitud caracteres inicializados  
  con el primer símbolo del alfabeto
```

```
REPETIR
    candidatosEvaluados <- candidatosEvaluados + 1

    SI SHA256(candidato) = hashObjetivo ENTONCES
        retornar encontrada, candidato, candidatosEvaluados

    avanzar candidato en representación de base |alfabeto|

HASTA que no sea posible avanzar

retornar no encontrada, candidatosEvaluados
```

5.1.6. Implementación en C++17.

La implementación está separada en:

- src/fb_password.hpp
- src/fb_password.cpp
- main.cpp

5.1.7. Construcción de casos de prueba

Se deben usar:

A1 = a..z, con 26 símbolos.

A2 = a..z, 0..9, con 36 símbolos.

A1: longitudes 3, 4, 5 y 6.

A2: longitudes 3, 4 y 5.

Caso base:

Alfabeto: A2, n = 5, Contraseña: abc12

Caso cadena concatenada:

angaritacastanomira

La semilla se calcula así:

$$\text{semilla} = \text{suma ASCII} \pmod{100000}$$

$$\text{semilla} = 2009$$

Luego genera cinco contraseñas usando:

$$x_0 = 2009$$

$$x_{i+1} = (1103515245 \cdot x_i + 12345) \pmod{2^{31}}$$

Alfabeto: A1, n = 4, Contraseña: hmbk

ENTREGA I

Alfabeto: A2, $n = 4$, Contraseña: bstm

Alfabeto: A1, $n = 5$, Contraseña: jofcp

Alfabeto: A2, $n = 5$, Contraseña: wli7k

Alfabeto: A1, $n = 6$, Contraseña: purovm

5.1.8. Ejecución con diferentes tamaños de entrada (longitudes y alfabetos).

Compilar:

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -Isrc src/main.cpp  
src/fb_password.cpp -o tests/build/Debug/fb_app.exe
```

Para casos del equipo de instancias privadas:

```
.\tests\build\Debug\fb_app.exe --generate-instances
```

Nota: Puede tardar la ejecución por la exigencia de FB.

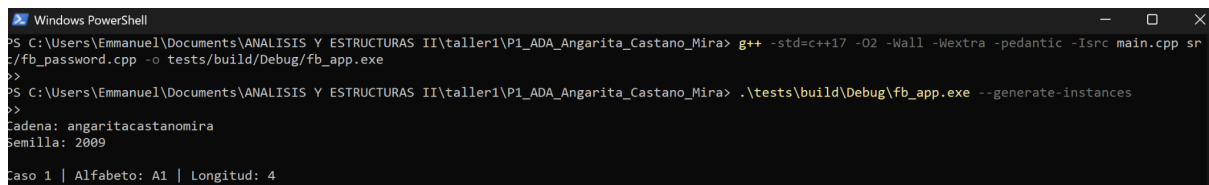
Luego para caso base:

- Caso base de prueba: A2, $n=5$, contraseña abc12

```
.\tests\build\Debug\fb_app.exe --validate
```

Para comparar con el diccionario sintético:

```
.\tests\build\Debug\fb_app.exe --dictionary HASH resources/diccionario.txt
```



```
Windows PowerShell
PS C:\Users\Emmanuel\Documents\ANALISIS Y ESTRUCTURAS II\taller1\P1_ADA_Angarita_Castano_Mira> g++ -std=c++17 -O2 -Wall -Wextra -pedantic -Isrc main.cpp sr
c/fb_password.cpp -o tests/build/Debug/fb_app.exe
>
PS C:\Users\Emmanuel\Documents\ANALISIS Y ESTRUCTURAS II\taller1\P1_ADA_Angarita_Castano_Mira> .\tests\build\Debug\fb_app.exe --generate-instances
>
Cadena: angaritacastanomira
Semilla: 2009
Caso 1 | Alfabeto: A1 | Longitud: 4
```

5.1.9. Medición del tiempo de ejecución para cada configuración.

El programa registra:

- Estado
- Texto plano
- Candidatos evaluados
- Tiempo (ms)

La medición utiliza: `std::chrono::steady_clock`

ENTREGA I

```
Caso 1 | Alfabeto: A1 | Longitud: 4
Contraseña: hmbk
Hash: b26ec142bc2a5875999ffdb0f59cb7f37463bb8b151c81869fea9e11882c8753
Metodo: fuerza bruta - caso 1
Estado: encontrada
Texto plano: hmbk
Candidatos evaluados: 131181
Tiempo (ms): 812.625
```

```
Caso 2 | Alfabeto: A2 | Longitud: 4
Contraseña: bstm
Hash: f907b2f91026c9336f730f0641991e5310a3d69b63310cf0c5c033387b6829fd
Metodo: fuerza bruta - caso 2
Estado: encontrada
Texto plano: bstm
Candidatos evaluados: 70681
Tiempo (ms): 450.692
```

```
Caso 3 | Alfabeto: A1 | Longitud: 5
Contraseña: jofcp
Hash: 9f1f96324be61dad331f9be1930be0ccbb1b9ae1b8cf7d93000ac93762f7e266
Metodo: fuerza bruta - caso 3
Estado: encontrada
Texto plano: jofcp
Candidatos evaluados: 4362296
Tiempo (ms): 28125.9
```

```
Caso 4 | Alfabeto: A2 | Longitud: 5
Contraseña: w1i7k
Hash: 5f64254251d780bc9778d4cb558b12c1d21a95c67f7929cec5bf4608e81f0da0
Metodo: fuerza bruta - caso 4
Estado: encontrada
Texto plano: w1i7k
Candidatos evaluados: 38222831
Tiempo (ms): 271575
```

```
Caso 5 | Alfabeto: A1 | Longitud: 6
Contraseña: purovm
Hash: 53aeaf14243cf4211f7628e68bf5ecfc068cb1ba962ce8d9ebe7b8c126ba34db
Metodo: fuerza bruta - caso 5
Estado: encontrada
Texto plano: purovm
Candidatos evaluados: 187668975
Tiempo (ms): 1.32393e+006
```

```
PS C:\Users\Emmanuel\Documents\ANALISIS Y ESTRUCTURAS II\taller1\P1_ADA_Angarita_Castano_Mira> .\tests\build\Debug\fb_app.exe -dictionary 8d51feb34e3e69f6
fa5dffc577e2c60498cf9a7fcd835f9f6af1505b71d74773 resources\diccionario.txt
Metodo: ataque por diccionario
Estado: encontrada
Texto plano: abc12
Candidatos evaluados: 4
Tiempo (ms): 1.992
Pertenece al diccionario: si
PS C:\Users\Emmanuel\Documents\ANALISIS Y ESTRUCTURAS II\taller1\P1_ADA_Angarita_Castano_Mira>
```

5.1.10. Análisis de resultados experimentales.

Los resultados muestran que el tiempo de ejecución aumenta al incrementar la longitud de la contraseña. Esto ocurre porque el espacio de búsqueda crece exponencialmente. Para una misma longitud, el alfabeto A2 requiere más evaluaciones que A1, debido a que contiene 36 símbolos frente a los 26 de A1. El número de candidatos evaluados depende de la posición de la contraseña generada dentro del orden de enumeración. Si aparece al inicio, la ejecución es rápida; si aparece al final, se aproxima al peor caso.

En el peor caso, la cantidad de candidatos corresponde a $|\Sigma|^n$. Por lo tanto, la complejidad temporal es: $O(n \cdot |\Sigma|^n)$

El comportamiento observado confirma que agregar símbolos al alfabeto o aumentar la longitud incrementa considerablemente el costo de la búsqueda. La memoria adicional se mantiene en $O(n)$ porque solo se almacena el candidato actual.

5.1.11. Análisis de complejidad, contrastando la cota teórica $|\Sigma|^n$ con el crecimiento medido.

Para un alfabeto de tamaño k y longitud n :

Tiempo por candidato: $O(n)$ para calcular SHA-256.

Cantidad de candidatos: k^n .

Complejidad total aproximada: $O(n \cdot k^n)$

Espacio adicional: $O(n)$

El crecimiento medido debe aproximarse al crecimiento exponencial teórico. A2 crece más rápidamente que A1 porque:

$$36^n > 26^n$$

5.1.12. Conclusiones del módulo.

La fuerza bruta garantiza encontrar la contraseña si pertenece al espacio explorado, porque enumera todos los candidatos exactamente una vez. Sin embargo, el costo aumenta exponencialmente con la longitud y el tamaño del alfabeto. SHA-256 permite verificar cada candidato, pero no reduce la cantidad de combinaciones que deben probarse. Por eso, este método resulta útil para experimentar y demostrar complejidad, pero no es adecuado para contraseñas largas o alfabetos grandes.

5.2. Backtracking:

5.2.1. Modelamiento del problema.

Alfabeto Definido en **BTDatos.hpp** como **ALFABETO_BT**, con 67 símbolos:

Clase	Caracteres	Cantidad
Minúsculas	a ... z	26
Mayúsculas	A ... Z	26
Dígitos	0 ... 9	10
Símbolos	! @ # \$ %	5
Total		67

Denotamos el alfabeto por Σ , con $|\Sigma| = 67$. Cada carácter pertenece a exactamente una de las cuatro clases disjuntas: minúsculas **L**, mayúsculas **U**, dígitos **D** y símbolos **S**.

Estado parcial. Un estado parcial es un prefijo $s = s_0 s_1 \dots s_{k-1} \in \Sigma^k$ de longitud k (en el código, $k = \text{pos}$, con $0 \leq k \leq n$). Para no re-escanear el prefijo en cada nodo, el estado se enriquece con cuatro contadores acumulados que resumen las clases ya colocadas:

$c_L(s) = \#\{i < k : s_i \in L\}$ (nLower)

$c_U(s) = \#\{i < k : s_i \in U\}$ (nUpper)

$c_D(s) = \#\{i < k : s_i \in D\}$ (nDigit)

$c_S(s) = \#\{i < k : s_i \in S\}$ (nSymbol)

Así, el estado del nodo es la tupla $(s, k, c_L, c_U, c_D, c_S)$. En la implementación, s es el buffer password, k es pos, y los cuatro contadores son nLower, nUpper, nDigit, nSymbol (la política pol y el par contador, limite acompañan la recursión pero son constantes/globales, no parte del estado del nodo).

Estado inicial. El prefijo vacío $s = \varepsilon$, es decir $k = 0$ y todos los contadores en 0. Es la raíz del árbol de búsqueda.

Estados terminales. Los prefijos de longitud completa, $k = n$ (las cadenas de Σ^n). Un terminal es solución si además satisface toda la política; gracias a la poda de factibilidad, todo prefijo que alcanza $k = n$ ya cumple los mínimos, por lo que el caso base solo emite y cuenta.

Política (derivada de la semilla). La política vive en el struct PoliticaBT. Sus cuatro mínimos provienen de la semilla del equipo (calculada en tests/verificarSemilla.cpp a partir de los apellidos ordenados). Los valores instanciados son:

minLower = 2 minUpper = 2

minDigit = 3 minSymbol =

suma de mínimos = 8

prohibirSeguidos = true(regla de no consecutivos),

Es decir, toda contraseña válida debe tener ≥ 2 minúsculas, ≥ 2 mayúsculas, ≥ 3 dígitos, ≥ 1 símbolo y ningún par de caracteres iguales adyacentes. La suma de mínimos, 8, es la longitud mínima factible: ninguna cadena más corta que 8 puede cumplir la política (dato clave para el caso $n = 6$).

5.2.2. Definición de entradas y salidas.

Entrada

- El alfabeto Σ (67 símbolos, fijo en ALFABETO BT)
- La longitud n de la cadena
- Los parámetros de la política: minLower, minUpper, minDigit, minSymbol y si se prohíben o no que hayan 2 caracteres seguidos.
- Un límite de cadenas a generar (controlar salida), el límite está establecido en 3500 cadenas.

aplica política alguna (todos los mínimos en 0, sin no-consecutivos).

Salida

Para cada n , esta implementación calcula el conjunto de contraseñas válidas— cada cadena se escribe, una por línea, en results/PasswordsBT.txt— acotado el límite: se emiten las primeras $\min(\#soluciones, 3500)$ contraseñas en el orden recorrido en profundidad. Adicionalmente, cada ejecución devuelve el conteo de cadenas emitidas (contador) y el tiempo de la corrida

5.2.3. Definición de las restricciones.

Cada regla de la política se formaliza como una condición sobre el prefijo s de longitud k , comprobable incrementalmente al extenderlo con un carácter c en la posición k :

- R1 — Pertenencia al alfabeto. $s_i \in \Sigma$ para todo i . Se garantiza por construcción: el bucle solo prueba caracteres $c \in \text{ALFABETO_BT}$.
- R2 — Longitud exacta. Una solución cumple $|s| = n$. Verificable en el terminal: $k = n$.
- R3 — No consecutivos (si prohibirSeguidos). $\forall i \in [1, k-1] : s_i \neq s_{i-1}$. Como el prefijo previo ya la respeta, al añadir c en la posición k basta la comprobación local: (en el código: $\text{pos} > 0 \ \&\& \ c == \text{password}[\text{pos}-1] \Rightarrow$ se descarta ese hijo).
 $k = 0 \ \vee \ c \neq s_{k-1}$
- R4 — Mínimos por clase. Una solución cumple $c_L(s) \geq \text{minLower}$, $c_U(s) \geq \text{minUpper}$, $c_D(s) \geq \text{minDigit}$, $c_S(s) \geq \text{minSymbol}$. Sobre un prefijo parcial esto no puede exigirse todavía (aún faltan caracteres), pero sí puede verificarse su factibilidad: definiendo el déficit el prefijo s puede extenderse a

ENTREGA I

una solución solo si faltan(s) $\leq n - k$ (los requisitos pendientes caben en las posiciones libres). La violación faltan(s) $> n - k$ es la condición verificable que poda la rama (Sección 4). En el terminal $k = n$ se tiene faltan(s) = 0, lo que equivale a que R4 se cumple con igualdad de garantía.

$$\begin{aligned} \text{faltan}(s) = & \max(0, \text{minLower} - c_L) + \max(0, \text{minUpper} - c_U) \\ & + \max(0, \text{minDigit} - c_D) + \max(0, \text{minSymbol} - c_S) \end{aligned}$$

5.2.4. Diseño del algoritmo.

La función de factibilidad faltan $> n - \text{pos}$

La poda central es puramente aritmética. En un prefijo cualquiera se calcula cuántos caracteres de cada clase todavía faltan para alcanzar los mínimos:

$$\begin{aligned} \text{faltan} = & \max(0, \text{minLower} - n\text{Lower}) \\ & + \max(0, \text{minUpper} - n\text{Upper}) \\ & + \max(0, \text{minDigit} - n\text{Digit}) \\ & + \max(0, \text{minSymbol} - n\text{Symbol}) \end{aligned}$$

faltan es una cota inferior del número de posiciones que aún deben "gastarse" obligatoriamente en clases deficitarias. Por otro lado, $n - \text{pos}$ es el número de posiciones libres que quedan por llenar.

La lógica matemática: si los requisitos pendientes exceden los espacios disponibles —faltan $> n - \text{pos}$ — entonces es imposible completar el prefijo cumpliendo la política, sin importar qué caracteres se coloquen después. En ese instante toda la rama es infactible y se poda con return. La cota es admisible (nunca poda una rama que sí tenía solución) porque cada unidad de faltan requiere consumir al menos una posición distinta.

Un corolario barato pero potente: en la raíz, con todos los contadores en 0, faltan = suma de mínimos = 8. Si $n < 8$, la desigualdad se viola de inmediato y el árbol entero se poda en el primer nodo (ver $n = 6$).

Contadores por valor (no se deshace manualmente)

Los contadores nLower, nUpper, nDigit, nSymbol se pasan por valor en cada llamada recursiva:

```
backtracking(password, pos + 1,  
              nLower + (esMinuscula(c) ? 1 : 0),... );
```

Esto es una decisión de diseño deliberada. En backtracking clásico se muta un estado compartido y hay que deshacer (--nLower) al retroceder; olvidar un undo es una fuente típica de errores. Al pasar los contadores por valor, cada nodo recibe su propia copia ya actualizada; cuando la recursión retorna, la copia se descarta sola y el marco del llamador conserva intactos sus valores. El "deshacer" es automático por semántica de la pila de llamadas. El costo

ENTREGA I

(copiar cuatro int) es despreciable frente a la claridad y la ausencia de errores de estado. El buffer password sí se comparte por referencia, pero cada posición se sobrescribe antes de usarse, así que no requiere undo explícito.

Estructura para los casos de prueba

main() fija un límite configurable MAX_CADENAS y recorre un arreglo casos[] de 5 pruebas distintas, cada una con su propia PolíticaBT completa (no solo variando n). Para cada caso:

1. Toma la PolíticaBT del caso (su n, sus cuatro mínimos y la bandera prohibirSeguidos).
2. Llama a generarPassword, que primero hace detección temprana de infactibilidad ($\text{minTotal} > n$) y, si procede, lanza el backtracking.
3. Mide el tiempo con clock() y reporta cadenas escritas y segundos, tanto a archivo (results/PasswordsBT.txt) como a consola.

Los 5 casos cubren desde la política de la semilla ($n \in \{8, 6, 10\}$) hasta una política casi vacía ($\text{minLower} = 1$) y una sin política alguna —donde la poda de factibilidad es nula y el recorrido degenera en fuerza bruta—, todos reutilizando la misma función backtracking sin cambios. La única pieza nueva es el struct CasoBT { const char* etiqueta; PolíticaBT pol; } que empaqueta etiqueta + política para el bucle.

5.2.5. Pseudocódigo.

```
Algoritmo BACKTRACKING(prefijo, pos, nL, nU, nD, nS, pol)
  Entrada: prefijo parcial, posición actual, contadores por clase, política
  Salida : emite (por efecto lateral) las contraseñas válidas, hasta 'limite'

  si contador ≥ limite entonces
    retornar // corte por límite de salida

  faltan ← max(0, pol.minL - nL) + max(0, pol.minU - nU)
           + max(0, pol.minD - nD) + max(0, pol.minS - nS)
  si faltan > pol.n - pos entonces
    retornar // PODA: rama infactible

  si pos = pol.n entonces // hoja: prefijo completo y válido
    Emitir(prefijo)
    contador ← contador + 1
    retornar

  para cada c en ALFABETO hacer // ramificación (≤ 67 hijos)
    si pol.prohibirSeguidos ∧ pos > 0 ∧ c = prefijo[pos-1] entonces
      continuar // poda regla no-consecutivos
    prefijo[pos] ← c
    BACKTRACKING(prefijo, pos+1,
                  nL + esMin(c), nU + esMay(c),
                  nD + esDig(c), nS + esSim(c), pol)
    si contador ≥ limite entonces retornar
  fin para
fin Algoritmo

Algoritmo GENERAR(pol, limite)
```

ENTREGA I

```
minTotal ← pol.minL + pol.minU + pol.minD + pol.minS
si minTotal > pol.n entonces           // detección temprana
    Reportar "longitud insuficiente"; retornar 0
contador ← 0
BACKTRACKING(prefijo_vacío, 0, 0,0,0,0, pol)
retornar contador
```

5.2.6. Implementación en C++17.

Poda anticipada por factibilidad

Se evalúa antes de ramificar, en cada nodo:

```
int faltan = 0;
if (pol.minLower - nLower > 0) faltan += pol.minLower - nLower;
if (pol.minUpper - nUpper > 0) faltan += pol.minUpper - nUpper;
if (pol.minDigit - nDigit > 0) faltan += pol.minDigit - nDigit;
if (pol.minSymbol - nSymbol > 0) faltan += pol.minSymbol - nSymbol;
if (faltan > pol.n - pos) return; // rama imposible -> retroceder
```

los if implementan el `max(0, ·)`: solo cuentan las clases deficitarias (si ya se cumplieron un mínimo, no se aportan a **faltan**).

Detección de la variante infactible ($n = 6$)

Antes de tocar el árbol, `generarPassword` compara la suma de mínimos contra n :

```
int minTotal = pol.minLower + pol.minUpper + pol.minDigit + pol.minSymbol; // = 8
if (minTotal > pol.n) {           // p.ej. n = 6 -> 8 > 6
    // reporta longitud insuficiente y retorna 0, sin explorar nada
    return 0;
}
```

Con la política base ($\text{minTotal} = 8$), la longitud $n = 6$ se descarta aquí mismo: 0 cadenas, sin recorrer el árbol. Aunque esta guarda no estuviera, la poda de factibilidad cortaría igualmente en la raíz ($\text{faltan} = 8 > 6 = n - 0$); la guarda solo lo hace explícito y produce un mensaje legible.

Regla de no-consecutivos durante la ramificación

```
for (char c : ALFABETO_BT) {
    if (pol.prohibirSeguidos && pos > 0 && c == password[pos - 1])
        continue;           // salta el hijo que repetiría el carácter previo
    password[pos] = c;
    backtracking(password, pos + 1, /* contadores actualizados por valor */ ...);
    if (contador >= limite) return;
}
```

La restricción se aplica en el momento de generar cada hijo (no se genera y luego se descarta), y el chequeo `contador >= limite` propaga el corte por límite hacia arriba en la pila.

5.2.7. Construcción de casos de prueba

ENTREGA I

Caso	n	min L	min U	min D	min S	Σm_i	no-cons ec.	Caden as	Naturaleza
C1	8	2	2	3	1	8	sí	3500	Factible, ajuste justo ($\Sigma = n$)
C2	6	2	2	3	1	8	sí	0	Infactible ($\Sigma > n$) → poda en raíz
C3	10	2	2	3	1	8	sí	3500	Factible, holgura 2
C4	8	1	0	0	0	1	no	3500	Solo $\text{minLower}=1$, sin otras políticas
C5	6	0	0	0	0	0	no	3500	Sin política alguna → poda nula = fuerza bruta

5.2.8. Ejecución con diferentes tamaños de entrada (longitudes y alfabetos).

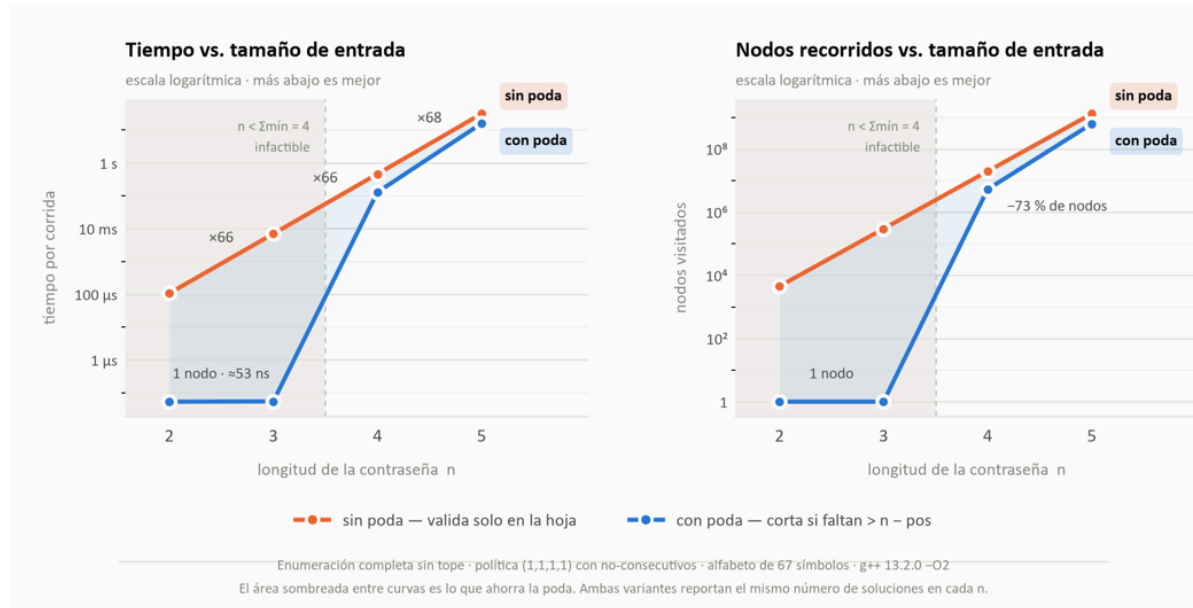
Caso	n	Política	Soluciones	Nodos visitados	Tiempo
C1 (ajuste justo)	8	(2,2,3,1) no-consec	3500	34 608	0.001 s
C2 (infactible)	6	(2,2,3,1) no-consec	0	1	~0.000 s
C3 (holgura 2)	10	(2,2,3,1) no-consec	3500	34 610	0.001 s
C4 (solo $\text{minLower}=1$)	8	(1,0,0,0) sin no-consec	3500	3 560	0.001 s
C5 (sin política)	6	(0,0,0,0) sin no-consec	3500	3 558	~0.000 s

cuando no hay política, la poda se anula y BT se vuelve fuerza bruta.

C1–C3 (política de la semilla) visitan ~34 600 nodos para juntar 3500 cadenas: la poda de factibilidad descarta subárboles deficitarios y hay que explorar más para encontrar cadenas que cumplan (2,2,3,1). En cambio C4 y

C5 visitan $\sim 3\,560$ nodos —casi el mínimo posible, ≈ 3500 hojas + su rama izquierda— precisamente porque la poda no interviene: al no haber mínimos (o solo uno trivial), toda hoja alcanzada es solución y el algoritmo emite las 3500 primeras casi sin ramificar. Es la firma empírica de la poda nula / fuerza bruta. Menos nodos bajo el tope, pero sin ninguna capacidad de recorte —el peor caso $\Theta(67^n)$ se manifestaría al quitar el tope.

5.2.9. Medición del tiempo de ejecución.



5.2.10. Análisis de resultados experimentales.

- C2 ($n = 6$, semilla) — 0 soluciones, poda casi total. Este no es un error, es el resultado correcto y esperado: $\Sigma_{\min} = 8 > 6 = n$, así que ninguna contraseña de longitud 6 puede cumplir la política. El algoritmo lo detecta en 1 solo nodo (poda en la raíz) o en la guarda temprana, sin explorar el árbol. Es la demostración empírica del mejor caso.
- C1 y C3 (factibles, semilla). Alcanzan el tope de 3500 cadenas visitando $\sim 34\,600$ nodos. El número de nodos es casi idéntico entre $n = 8$ y $n = 10$ porque la búsqueda en profundidad encuentra las primeras 3500 soluciones "por la izquierda" del árbol y se corta; la n mayor apenas añade profundidad a esas mismas ramas.
- C4 ($n = 8$, solo $\minLower = 1$). Solo 3 560 nodos: al no exigirse mayúsculas, dígitos ni símbolos, y con el único mínimo $\minLower = 1$ trivial de cumplir, la poda de factibilidad prácticamente no interviene y las 3500 primeras hojas se emiten casi sin ramificar. Es ya una política de poda casi nula.
- C5 ($n = 6$, sin política) — poda nula = fuerza bruta. También $\sim 3\,560$ nodos (3 558) bajo el tope de 3500: como toda cadena es válida y faltan = 0 siempre, ninguna rama se poda y el recorrido es una enumeración exhaustiva de Σ^n cortada en 3500. Aquí aparece el matiz crucial del enunciado: con tope de soluciones la política vacía termina antes (menos nodos), pero eso no

ENTREGA I

contradice su peor caso teórico: sin el tope, C5 recorrería el árbol completo $67^6 \approx 9.0 \times 10^{10}$. La escasez de nodos bajo el tope es justamente la señal de que no hubo poda: se emitieron las primeras hojas que se encontraron, sin descartar nada.

5.2.11. Conclusiones del módulo.

6. Diseño algorítmico (decisiones de diseño de cada algoritmo).

Fuerza Bruta: Se diseñaron dos algoritmos: fuerza bruta, que genera sistemáticamente todas las cadenas posibles por longitud y alfabeto, calculando su SHA-256 hasta encontrar coincidencia; y ataque por diccionario, que evalúa secuencialmente cada palabra del archivo. Para evitar un consumo excesivo de memoria, ambos procesan un candidato a la vez y detienen la búsqueda al hallar la contraseña o agotar el espacio.

BT: Apartado 5.2.4

7. Pseudocódigo (ambos módulos).

Fuerza Bruta: Apartado 5.1.5.

BT: Apartado 5.2.5

8. Implementación

Fuerza Bruta: La solución se implementó en C++17 mediante módulos separados (fb_password.hpp y fb_password.cpp), manteniendo la generación de candidatos bajo demanda para usar espacio $O(n)$. La enumeración incrementa la cadena como un número en base $|\Sigma|$, evitando repeticiones, mientras que cada candidato se verifica mediante SHA-256. Además, se incorporaron lectura secuencial del diccionario, validación de hashes, conteo de candidatos y medición con `std::chrono`; `main.cpp` centraliza los comandos de ejecución sin alterar la lógica de Backtracking.

BT: Apartado 5.2.6

9. Análisis de complejidad (temporal y espacial).

Fuerza Bruta: Apartado 5.1.11.

BT: Apartado 5.2.8

10. Casos de prueba

Fuerza Bruta: Apartado 5.1.7.

BT: Apartado 5.2.8

11. Experimentación

Fuerza Bruta:

Tabla de ejecución:

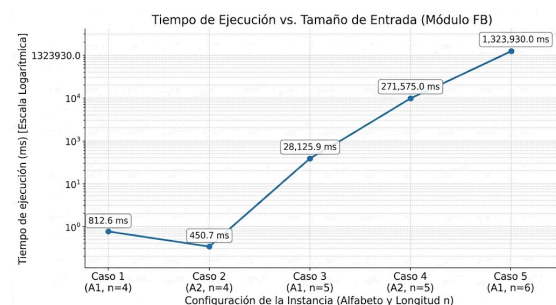
Caso	Alfabeto (Σ)	Longitud (n)	Candidatos Evaluados	Tiempo (ms)
1	A1 (26)	4	131181	812.625
2	A2 (36)	4	70681	450.692
3	A1 (26)	5	4362296	28125.9
4	A2 (36)	5	38222831	271575.0
5	A1 (26)	6	187668975	1323930.0

Gráfica tiempo de ejecución vs. tamaño de entrada:

Se logra observar que el crecimiento empírico validó la complejidad exponencial teórica, multiplicando el tiempo de ejecución en un factor proporcional al tamaño del alfabeto al incrementar la longitud, con mínimas desviaciones por sobrecarga del procesador. Por su parte, el punto de intratabilidad computacional se alcanzó en la longitud 6 para el alfabeto A1 y en la longitud 5 para el A2, evidenciando que dimensiones superiores exigen tiempos inmanejables para la experimentación local.

Comparación: Fuerza Bruta vs. ataque por diccionario

Caso 1: Hash:



b26ec142bc2a5875999ffdb0f59cb7f37463bb8b151c81869fea9e11882c8753

ENTREGA I

```
PS C:\Users\Emmanuel\Documents\ANALISIS Y ESTRUCTURAS II\taller1\PI_ADA_Angarita_Castano_Mira> .\tests\build\Debug\fb_app.exe --dictionary b26ec142bc2a587599
9ffdb0f59cb7f37463bb8b151c81869fea9e11882c8753 resources/diccionario.txt
Metodo: ataque por diccionario
Estado: No encontrado dentro del espacio
Candidatos evaluados: 500
Tiempo (ms): 5.929
Pertenece al diccionario: no
```

Se observa que, el ataque por diccionario es sumamente rápido al evaluar una lista finita, pero falla ante contraseñas aleatorias. En contraste, la fuerza bruta garantiza encontrar la contraseña, pero su costo temporal crece de forma exponencial hasta volverse inviable.

Caso 2: Hash: f907b2f91026c9336f730f0641991e5310a3d69b63310cf0c5c033387b6829fd

Caso 3: Hash: 9f1f96324be61dad331f9be1930be0ccbb1b9ae1b8cf7d93000ac93762f7e266

Caso 4: Hash: 5f64254251d780bc9778d4cb558b12c1d21a95c67f7929cec5bf4608e81f0da0

Caso 5: Hash: 53aeaf14243cf4211f7628e68bf5ecfc068cb1ba962ce8d9ebe7b8c126ba34db

Nota: Al ejecutar los demás Hashes, se repite el mismo resultado ya resaltado.

BT: Apartado 5.2.9

12. Resultados.

Fuerza Bruta: 5.1.10.

BT: mirar results/PasswordsBT.txt y /results/BTExperimento.csv

13. Análisis de resultados (contraste teórico vs. empírico).

Los resultados de Fuerza Bruta confirman lo que se esperaba: el espacio de búsqueda crece de forma exponencial, siguiendo la relación $|\Sigma|^n$. Esto significa que, con aumentar un poco la longitud de la contraseña o el tamaño del alfabeto, la cantidad de candidatos se dispare. En A1, por ejemplo, se pasó de 131.181 candidatos evaluados para longitud 4, a 4.362.296 para longitud 5, y a 187.668.975 para longitud 6. En A2, donde el alfabeto es de 36 caracteres, se evaluaron 70.681 candidatos en longitud 4 y 38.222.831 en longitud 5.

Los tiempos de ejecución van de la mano con este crecimiento. En A1, se observó un salto de unos 812,625 ms en longitud 4 a 28.125,9 ms en longitud 5, y hasta 1.323.930 ms en longitud 6. En A2 los tiempos fueron de 450,692 ms para longitud 4 y 271.575 ms para longitud 5.

Vale aclarar que las diferencias entre instancias de una misma longitud no dependen del algoritmo en sí, sino de dónde cae la contraseña dentro del orden de enumeración: como la búsqueda se detiene apenas la encuentra, si aparece temprano el tiempo es menor, y si aparece tarde, mayor.

Con Backtracking la historia cambia un poco. El peor caso sigue siendo exponencial, pero la poda ayuda bastante a reducir los estados que realmente se exploran. El algoritmo corta una rama en dos situaciones: cuando ya no alcanzan las posiciones disponibles para cumplir con la política de la contraseña, o cuando se generarían caracteres consecutivos repetidos. Para $n=2$ y $n=3$ directamente no hay nada que explorar, porque no se llega al mínimo de 8 caracteres exigido, así que la búsqueda se descarta desde la raíz.

ENTREGA I

Los números respaldan el beneficio de podar. Para $n=4$, la versión con poda visitó 5.248.322 nodos, frente a los 19.558.574 que visitó la versión sin poda; para $n=5$, la diferencia fue de 624.715.874 contra 1.290.865.886. Eso se traduce en una reducción de aproximadamente 73,16 % y 51,60 % de los nodos explorados, respectivamente. La poda no cambia la complejidad asintótica en el peor caso, pero sí muestra un ahorro real y considerable de trabajo en las instancias evaluadas.

14. Comparación algorítmica (fuerza bruta vs. diccionario; con poda vs. sin poda).

Fuerza Bruta y el ataque por diccionario se diferencian sobre todo en el tamaño de su espacio de búsqueda. Fuerza Bruta genera de forma sistemática todas las combinaciones posibles según el alfabeto y la longitud definidos, calculando el hash SHA-256 de cada una; si la contraseña está dentro de ese espacio, tarde o temprano la encuentra, aunque el costo puede ser exponencial. El ataque por diccionario, en cambio, simplemente recorre una lista de palabras y compara sus hashes con el objetivo. Al trabajar sobre un espacio mucho más chico, suele ser bastante más rápido cuando la contraseña está en esa lista, pero tiene una limitación clara: si no está, jamás la va a encontrar. Los experimentos del proyecto reflejan justo esa tensión entre velocidad y cobertura.

Comparar Backtracking con poda y sin poda deja ver otra diferencia interesante. Las dos variantes llegan exactamente al mismo conjunto de soluciones, pero la que usa poda se ahorra explorar ramas que de entrada no pueden llevar a una contraseña válida. Para $n=4$, ambas encontraron 811.200 soluciones, aunque la versión con poda visitó 5.248.322 nodos frente a los 19.558.574 de la versión sin poda. Para $n=5$, ambas también coincidieron en 132.631.200 soluciones, pero con poda se visitaron 624.715.874 nodos, contra 1.290.865.886 sin poda. Es decir, se llega al mismo resultado explorando muchísimo menos.

Esa diferencia se nota también en el tiempo. Para $n=4$, Backtracking con poda tardó cerca de 0,1283 segundos, frente a 0,4601 segundos sin poda. Para $n=5$, los tiempos fueron de aproximadamente 15,94 segundos con poda contra 31,46 segundos sin ella. El beneficio se hace más evidente justamente cuando las restricciones permiten detectar temprano que una rama no tiene sentido seguir explorándola.

Ahora bien, cuando no hay restricciones relevantes que aplicar, Backtracking termina pareciéndose a una enumeración exhaustiva, porque casi no hay ramas que se puedan descartar. Por eso la poda no cambia el peor caso teórico, que sigue siendo exponencial, aunque en la práctica sí genera mejoras sustanciales. En resumen: Fuerza Bruta apuesta por la cobertura total, el diccionario prioriza la velocidad sobre un conjunto acotado, y Backtracking saca provecho de las restricciones para reducir el trabajo real sin sacrificar el resultado.

15. Conclusiones.

Los experimentos confirmaron que Fuerza Bruta se comporta tal como predice la teoría: crece de forma exponencial. A mayor longitud de contraseña o tamaño de alfabeto, más candidatos y más tiempo de ejecución, al punto de volverse inviable incluso con tamaños relativamente pequeños. Eso sí, mientras la contraseña esté dentro del espacio explorado, el método garantiza encontrarla.

El ataque por diccionario resultó mucho más rápido cuando la contraseña está en la lista de palabras usada, pero tiene un límite claro: si no está ahí, no hay forma de encontrarla. Esto resume bien el trade-off entre los dos enfoques: Fuerza Bruta explora todo de forma exhaustiva, mientras que el diccionario acota el espacio de búsqueda a costa de dejar fuera posibles soluciones.

Con Backtracking se vio que la poda reduce bastante el trabajo sin cambiar las soluciones encontradas: menos nodos visitados, menos tiempo, y además permite descartar de entrada los casos imposibles (por ejemplo, cuando la longitud no alcanza para cumplir los requisitos mínimos).

En conclusión, no hay un método mejor en abstracto: todo depende del problema. Fuerza Bruta conviene cuando se necesita cubrir todo el espacio, y Backtracking rinde mejor cuando hay restricciones que permiten podar ramas a tiempo. Ambos siguen teniendo un peor caso exponencial, pero aprovechar bien las restricciones marca una diferencia real en la práctica.

16. Referencias.

Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer.
<https://doi.org/10.1007/978-3-030-54256-6>

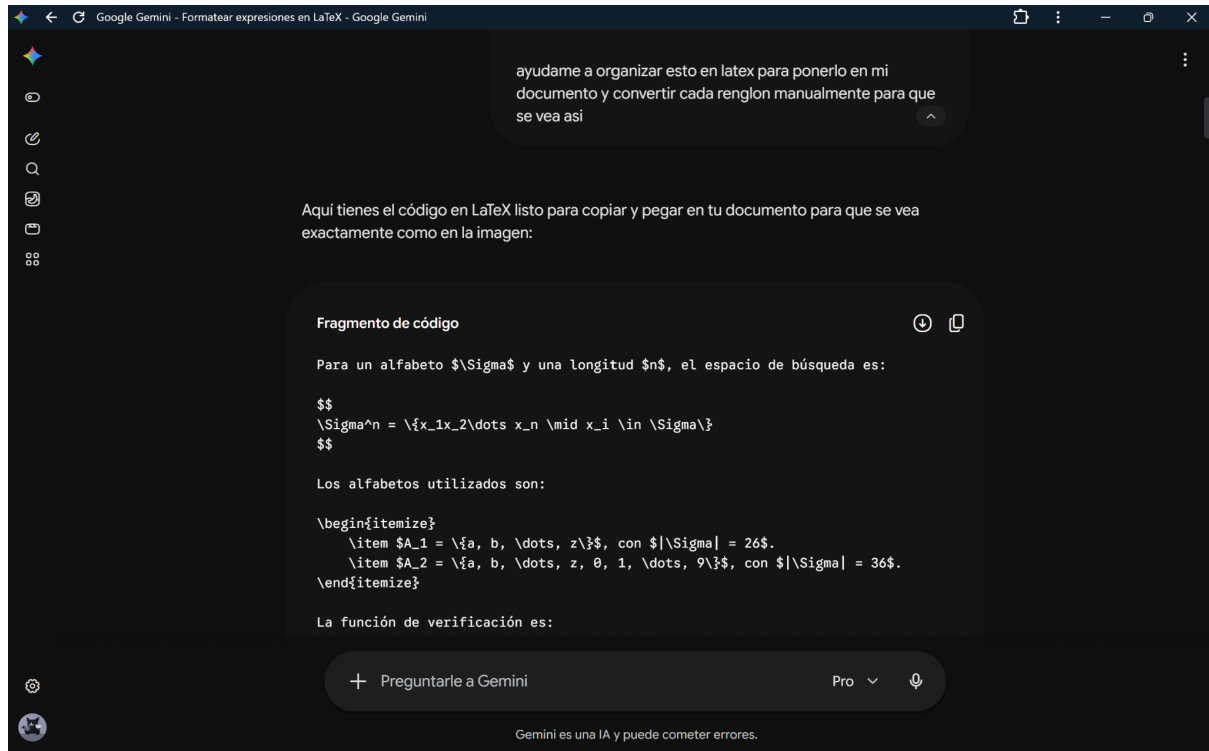
17. Uso de herramientas de IA (Sección 14).

A lo largo del desarrollo se utilizaron las herramientas de Inteligencia Artificial **Google Gemini** y **Claude (Anthropic)** como apoyo complementario, específicamente en los siguientes aspectos:

- **Mejora de redacción:** revisión de claridad, coherencia y estilo del texto, sin alterar el contenido técnico ni los resultados obtenidos.
- **Conversión a LaTeX:** apoyo en la transcripción y formateo del contenido al lenguaje LaTeX, incluyendo estructura de secciones, ecuaciones y referencias.
- **Sugerencias en el desarrollo:** recomendaciones puntuales sobre organización y enfoque de ciertas secciones del trabajo.

ENTREGA I

Todo el análisis, las decisiones técnicas, los resultados y las conclusiones presentadas en este informe son responsabilidad exclusiva del(los) autor(es), quien(es) revisó y validó el contenido generado o sugerido por dichas herramientas antes de su inclusión.



18. Contribución individual de los integrantes (Sección 13).

Diego Angarita: Backtracking - Diseño del algoritmo de backtracking (definición de estados, poda y condiciones de retroceso), implementación en código, construcción de casos de prueba para el módulo, análisis de complejidad temporal/espacial del enfoque, y redacción de la sección del informe correspondiente al backtracking (metodología y resultados).

Emmanuel Castaño: Fuerza Bruta - (en conjunto con Luis Miguel Mira) Diseño e implementación conjunta del algoritmo de fuerza bruta, participación en la construcción de casos de prueba y experimentación del módulo, análisis de complejidad, y redacción conjunta de la sección del informe correspondiente a fuerza bruta.

Luis Miguel Mira: Fuerza bruta (en conjunto con Emmanuel Castaño) Diseño e implementación conjunta del algoritmo de fuerza bruta, participación en la construcción de casos de prueba y experimentación del módulo, análisis de complejidad, y redacción conjunta de la sección del informe correspondiente a fuerza bruta.

Nota sobre el módulo de fuerza bruta (trabajo en pareja):

El módulo de Fuerza Bruta fue desarrollado en conjunto por Emmanuel Castaño y Luis Miguel Mira, mediante trabajo en pareja (*pair programming*): diseño, implementación, casos de prueba y análisis se realizaron de forma simultánea y colaborativa, no dividida por partes independientes. Por acuerdo del equipo, los commits correspondientes a este módulo quedaron registrados bajo la cuenta de GitHub ecastanos (Emmanuel Castaño), lo cual no refleja una menor contribución de Luis Miguel Mira, sino la dinámica de trabajo conjunto adoptada para esta parte del proyecto.